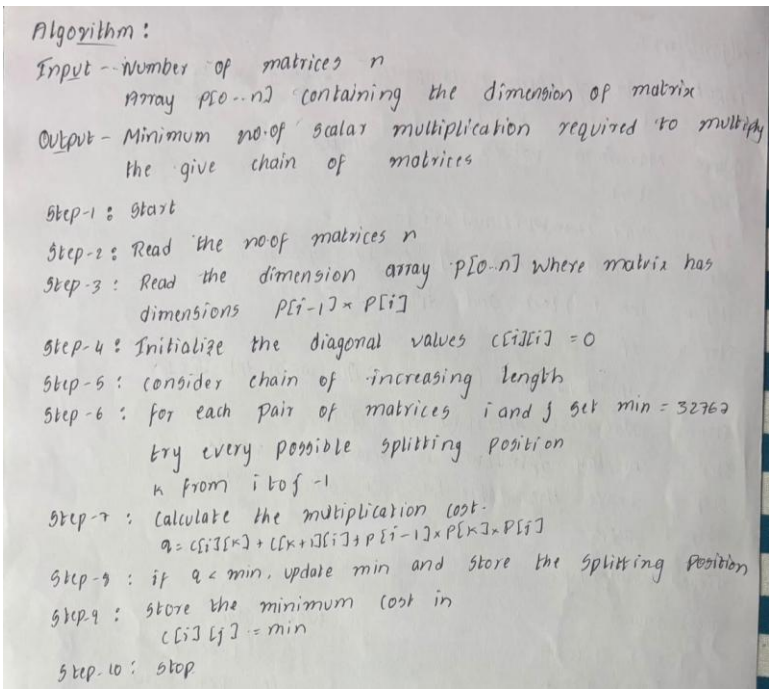
PRACTICAL – 6

AIM: Implementation of chain matrix multiplication using dynamic programming.

ALGORITHM:


Algorithm :
Input - Number of matrices n
Array p[0..n] containing the dimension of matrix
Output - Minimum no. of scalar multiplication required to multiply the give chain of matrices
Step-1 : Start
Step-2 : Read the no of matrices n
Step-3 : Read the dimension array p[0..n] where matrix has dimensions p[i-1] x p[i]
Step-4 : Initialize the diagonal values c[i][i] = 0
Step-5 : consider chain of increasing length
Step-6 : for each pair of matrices i and j set min = 32767
try every possible splitting position
k from i to j -1
Step-7 : Calculate the multiplication cost.
q = c[i][k] + c[k+1][j] + p[i-1] x p[k] x p[j]
Step-8 : if q < min, update min and store the splitting position
Step-9 : store the minimum cost in
c[i][j] = min
Step-10 : Stop

PROGRAM:

```
#include <iostream>
using namespace std;

int main() {
   int n, i, j, k, d, q, min;
   int p[20], c[20][20], k_table[20][20];

   cout << "Enter number of matrices: ";
   cin >> n;

   cout << "Enter the dimensions:" << endl;
   for (i = 0; i <= n; i++) {
      cin >> p[i];
   }

   for (i = 1; i <= n; i++) {
      c[i][i] = 0;
```

```
    }

    for (d = 1; d <= n - 1; d++) {
      for (i = 1; i <= n - d; i++) {
        j = i + d;
        min = 32767;

        for (k = i; k <= j - 1; k++) {
          q = c[i][k] + c[k + 1][j]
            + p[i - 1] * p[k] * p[j];

          if (q < min) {
            min = q;
            k_table[i][j] = k;
          }
        }

        c[i][j] = min;
      }
    }

    cout << "\nMinimum number of multiplications = "
       << c[1][n] << endl;

    return 0;
  }
```

OUTPUT:

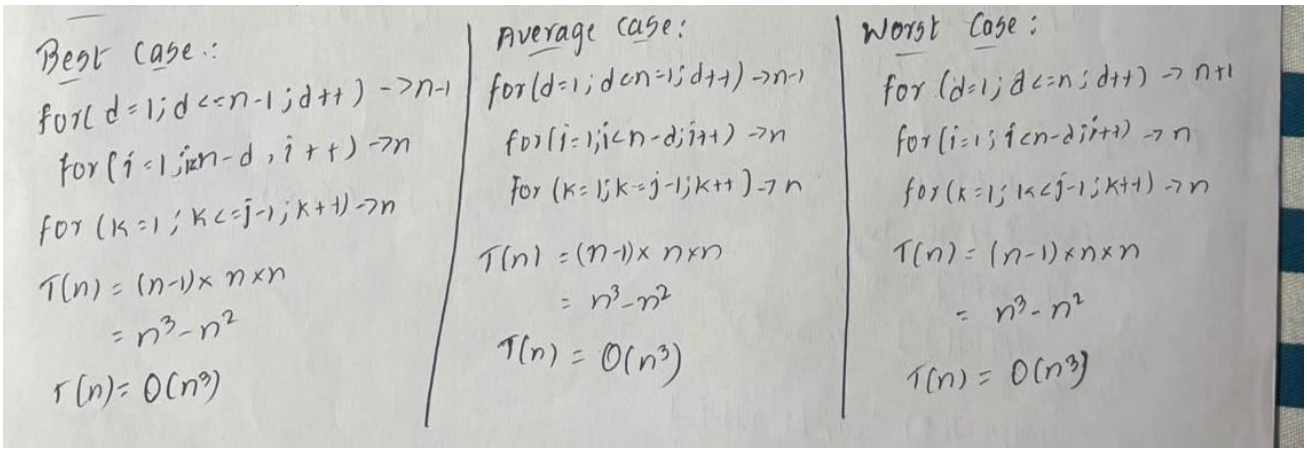
```
Output

Enter number of matrices: 4
Enter the dimensions:
10 20
20 30
30 40

Minimum number of multiplications = 19000


=== Code Execution Successful ===
```

TIME COMPLEXITY:



CONCLUSION:

The implementation and time analysis of Matrix Chain Multiplication using Dynamic Programming helps us understand its efficiency. The algorithm builds a DP table and considers different ways of splitting the matrix chain to find the minimum number of scalar multiplications. It has a time complexity of $O(n^3)$ in the best, average, and worst cases. Therefore, Dynamic Programming provides an efficient and systematic solution for solving the Matrix Chain Multiplication problem.